

Hyper-Parameter Optimization

Karanpreet Benipal

Introduction

This project takes a look at some current Hyper-Parameter Optimization (HPO) techniques, presents the algorithms and R implementations and results. The HPO problem is a potentially high dimensional non-convex optimization problem without a gradient. This presents many challenges. Research has become more relevant and prevalent in the last decade due to Neural Networks. The neural network hyper-parameters are more significant, where as with typical bagging and boosting methods default parameters are usually pretty good, with small improvements with adjusting the hyper-parameters. Neural Networks vary in size and structure which makes formulating their architecture as an optimization problem a challenge, to which there are multiple possible solutions.

Due to computational resource limitations, this project does not perform an exhaustive evaluation of all HPO algorithms or rigorous bench-marking. This intended to be a brief exploration of the HPO problem and a few solutions to said problem.

Optimizers

This project tests five different Hyper-Parameter Optimization techniques. This is a sample of the numerous solutions available. I chose ones that were interesting, commonly used, or particularly novel. In this section I will present a description of each along with pseudocode, strengths and weaknesses, and some extensions and considerations. The R code for each is in the appendix, each section has a link to the corresponding code. The R code is not the same as the one used in the tests, but rather has some features (such as runtime calculations and test errors) removed, for readability and to make the code more practical. The code for the objective functions are also omitted here. The full R script used and the rmd file to create this pdf are available on my Github [here](#).

Grid Search

Grid search involves manually constructing a grid over the search space, and evaluating each point to find the best one. It is neither efficient nor effective, we will make use of it as a benchmark, against which we compare all other algorithms.

Algorithm 1 Grid Search

```
1: procedure GRIDSEARCH(grid_points,  $\mathcal{D}_{\text{train}}$ )
2:    $T \leftarrow \text{expand.grid}(\text{grid\_points})$ 
3:   for  $i \in \{1, \dots, \text{length}(T)\}$  do
4:      $L_i \leftarrow \text{objective\_function}(\mathcal{D}_{\text{train}}, T_i)$ 
5:   end for
6:   return Parameter configuration with the lowest error.
7: end procedure
```

Considerations

Strengths:

- Simple to implement.

Weaknesses:

- Tests a large number of suboptimal configurations
- Does not effectively search the space, good configurations could lie in between predefined grid points.
- An effective grid may require prior knowledge, in which case a manual search could be more efficient.

Extensions:

- It is strongly not recommended to make use of grid search. Other algorithms are simply better.

Here is an example of what the parameter grid points were for Random Forest:

```
rf_params <- list( # 6 x 6, so a total of 36 models were checked
  m = c(0.5, 0.6, 0.7, 0.8, 0.9, 1),
  depth = 1:6
)
```

R code: [Click Me!](#)

Random Search

Random search involves generating n points in the search space by drawing from a joint distribution, marginal distributions for each parameter, or a mix of the two. In the implementation presented here each hyper-parameter has its' own distribution and the each parameter is drawn independently of the others. A main benefit is that less time is wasted on unimportant parameters. Consider this image:

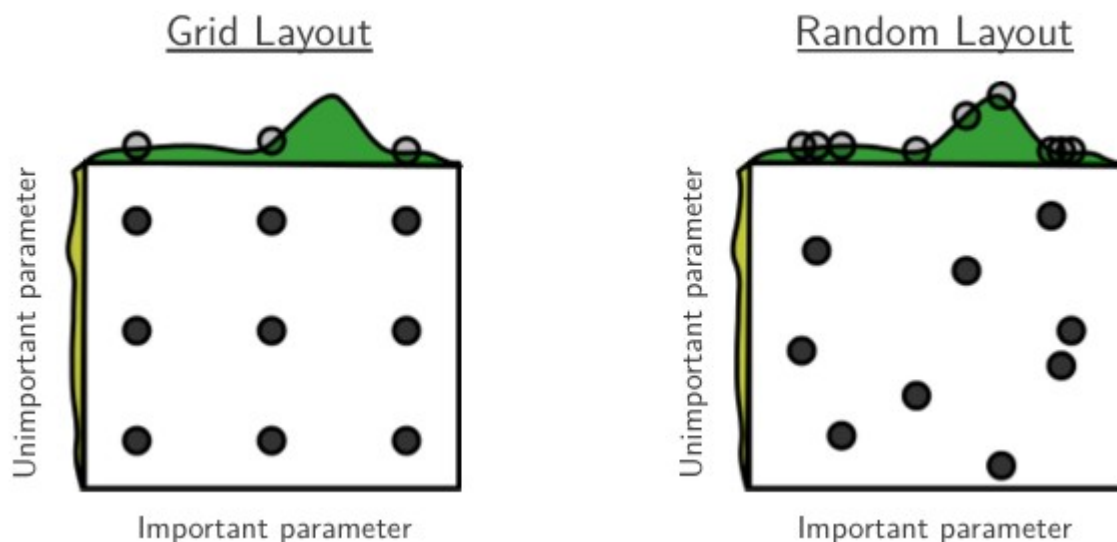


Figure 1: At the top and left are the marginal distributions. Random Search understands the important parameter far better than Grid Search, while not wasting time and resources on the unimportant parameter. Source: Figure 1 from “Random Search for Hyper-Parameter Optimmmization” by Bergstra & Bengio

Algorithm 2 Random Search

```
1: procedure RANDOMSEARCH( $n$ ,  $\text{dists}$ ,  $\mathcal{D}_{\text{train}}$ )
2:    $T \leftarrow \text{draw.points}(n, \text{dists})$ 
3:   for  $i \in \{1, \dots, n\}$  do
4:      $L_i \leftarrow \text{objective\_function}(\mathcal{D}_{\text{train}}, T_i)$ 
5:   end for
6:   return Parameter configuration with the lowest error.
7: end procedure
```

Considerations

Strengths:

- Not constrained to a grid and so can explore the full hyper-parameter space.
- The distribution can be particular to the parameter, leading to a more efficient search.

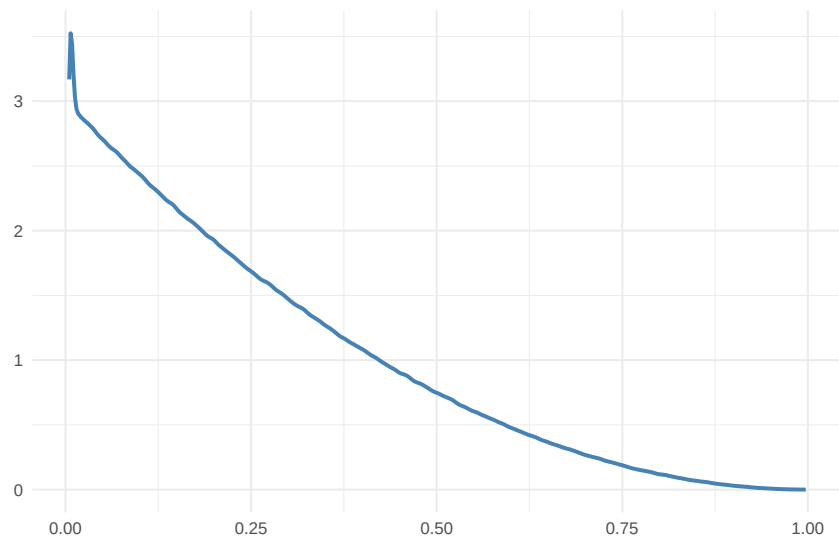
Weaknesses:

- Due to the stochastic nature of the algorithm a low number of iterations may not find a decent model as the space would not be covered effectively. Especially in higher dimensions as the curse of dimensionality is a factor.
- The algorithm may evaluate multiple configs that are very similar, while some parts of the parameter space remain unexplored

Here is an example of what was used for XGBoost, note that every element in the list is a function of n which provides n draws from the corresponding hyper-parameter distribution.

```
xgb_dists <- list(  
  # eta is min of 3 uniforms. favours smaller values, does not allow smaller than 0.005  
  eta = function(n) pmax(pmin(runif(n), runif(n), runif(n)), 0.005),  
  depth = function(n) sample(1:6, n, TRUE), # Integer between 1 and 6 (inclusive)  
  subsample = function(n) runif(n, 0.7, 1), # Uniform between 0.7 and 1  
  colsample = function(n) runif(n, 0.7, 1) # Uniform between 0.7 and 1  
)
```

Here's a look at the distribution for η :



This is useful as typically a small learning rate (η) is preferred.

R code: [Click Me!](#)

Bayesian Optimization using Gaussian Processes

Bayesian Optimization is an entire topic that could fill books (and it does). In brief, the idea is to select the next point of interest based on the current points we have evaluated. Now that we have new information we select the most interesting point again. We iterate on this until we reach some stopping criteria. In this implementation, the search would stop after: a) convergence, b) a maximum number of iterations is reached, or c) the time limit expired.

Consider this image:

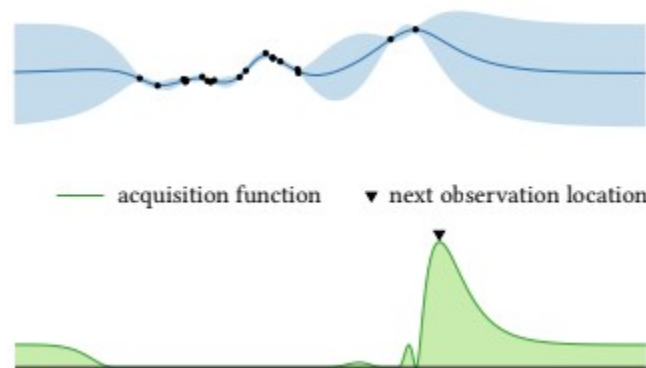


Figure 2: Gaussian Process confidence intervals with acquisition function. Source: Figure 4.12 from Roman Garnett's book "Bayesian Optimization".

The idea is to first evaluate some initial points and then to fit some surrogate model to those points. In the image above (and my implementation) Gaussian Process Regression is used. Then, some acquisition function is applied. Whichever point maximizes this acquisition function is the next point we are going to evaluate. Then simply repeat the process. Since we are minimizing, I have used the negative of the objective function here, since the base algorithm is a maximizer, but you could adjust the algorithm to minimize just as easily.

Considerations

Strengths:

- Learns from previous iterations to, after sufficient iterations, find better and better configurations.

Weaknesses:

- Fitting the GP and maximizing it can take a long time, especially in higher dimensions.
- Maximizing the GP can fail to converge, which is wasted time and effort.

Extensions:

- Consider using a grid or an Uniform Design (more below with SeqUD) as the initial grid. This will explore the space and then we can use the costly Bayes optimization to fine tune. This way we will need fewer iterations if we just start with a good understand of the objective function's surface.
- Instead of GP, one can use really any model for the response surface. Random Forest and Kernel density estimates are also common choices. The Kernel does suffer from the curse of dimensionality in higher dimensions, however Random Forest allegedly works very well, much better than GP in >20 dimensions.

Algorithm 3 Bayesian Optimization

```
1: procedure BAYESOPT(bounds, n,  $t_{max}$ ,  $\mathcal{D}_{train}$ )
2:    $n\_initial \leftarrow d + 1$ , for a d-dimensional search
3:    $T \leftarrow uniform\_points(bounds, n\_initial)$ 
4:   for  $i \in \{1, \dots, n\_initial\}$  do
5:      $T.error_i \leftarrow objective\_function(\mathcal{D}_{train}, T_i)$ 
6:   end for
7:   Begin Bayesian Optimization
8:   for  $j \in \{n\_initial + 1, \dots, n\}$  do
9:      $fit \leftarrow GaussianProcessRegression(T)$ 
10:     $x_{next} \leftarrow ArgMax(AcquisitionFunction(fit))$ 
11:     $x_{next}.error_i \leftarrow objective\_function(\mathcal{D}_{train}, x_{next})$ 
12:     $T \leftarrow append(x_{next}, T)$ 
13:    if SpottingCriteriaMet() then
14:      exit
15:    end if
16:  end for
17:  return Parameter configuration with the lowest error.
18: end procedure
```

R code: [Click Me!](#)

Hyperband

Hyperband involves strategically allocating resources to search more parameter combinations compared to random search in the same time. The paper suggests a few different ideas for what the resource could be: training time, dataset subsample, feature subsample. Different types of data would be more conducive to different resources. In this implementation dataset subsample was used, as that works with the different datasets and objective functions used in the tests.

Hyperband involves Successive Halving, which is similar to random search, it generates random points but evaluates them with a limited resource allocation. Then it chooses the best models and then increases the resource allocation and refits with the chosen points. The number of points is reduced by a factor of η each round.

Hyperband runs several brackets of Successive Halving with different starting resource allocations, so some brackets explore many different configurations and some spend more resources ‘exploiting’ a few configurations.

Considerations:

Strengths:

- Considers more configurations than Random Search in the same amount of time.

Weaknesses:

- We cannot control how resources are allocated, only how many there are. At least in the base algorithm. It seems rather arbitrary, the paper does not explain why.
- If the results of a low resource evaluation has low variance, then we do not need to repeatedly test that config multiple times. If it has high variance, then good configs may be discarded early.

Extensions:

- Try a more flexible approach, specifying the brackets manually may be preferred.
- For a specific dataset and model, one can test if low resources still result in accurate assessments and then use a higher η , or specify brackets manually to check more configurations.

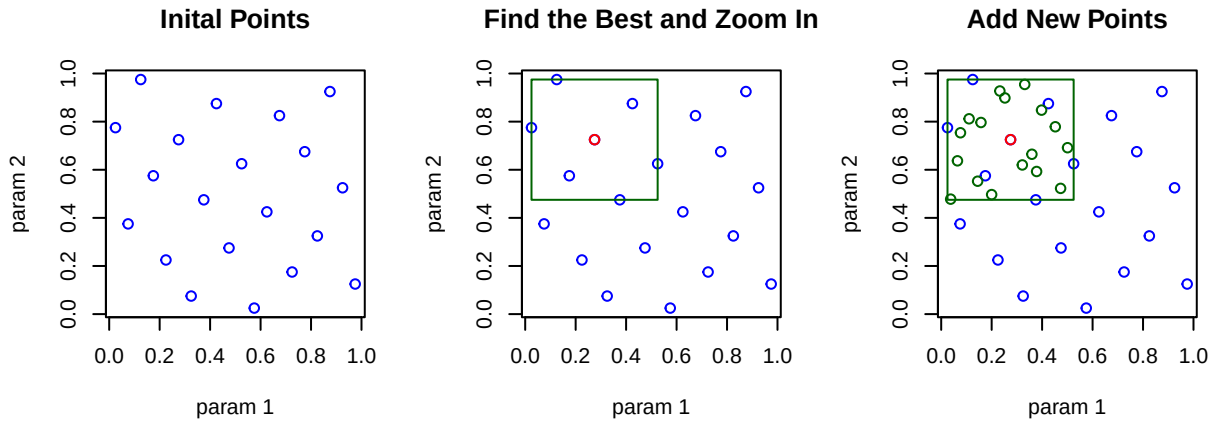
Algorithm 4 Hyperband

```
1: procedure HYPERBAND( $R, \eta, \text{dists}, \mathcal{D}_{\text{train}}$ )
2:    $S_{\text{max}} \leftarrow \lfloor \log_{\eta}(R) \rfloor$ 
3:    $B \leftarrow (S_{\text{max}} + 1)\hat{R}$ 
4:   for  $s \in \{S_{\text{max}}, \dots, 0\}$  do
5:      $n \leftarrow \lceil \frac{B\eta^s}{R \cdot (s+1)} \rceil$ 
6:      $r \leftarrow R\eta^{-s}$ 
7:      $T \leftarrow \text{draw.points}(n, \text{dists})$ 
8:     for  $i \in \{0, \dots, s\}$  do
9:        $n_i \leftarrow \lfloor n\eta^{-i} \rfloor$ 
10:       $r_i \leftarrow r\eta^i$ 
11:      for  $j \in \{1, \dots, n\}$  do
12:         $L_j \leftarrow \text{objective\_function}(\mathcal{D}_{\text{train}}, T_j, r_j)$ 
13:      end for
14:       $T \leftarrow \text{top\_k}(T, L, \lfloor \frac{n_i}{\eta} \rfloor)$ 
15:    end for
16:  end for
17:  return Parameter configuration with the lowest error.
18: end procedure
```

R code: [Click Me!](#)

Sequential Uniform Designs (SeqUD)

SeqUD can be thought of as a “batch-sequential” algorithm. Unlike Bayes where one config at a time is selected, here we select many configs within a smaller and smaller search space. The first step is selecting points in the search space that fill the full parameter space as evenly as possible. This is determined by a discrepancy measure, this implementation uses the ‘wrap-around discrepancy’. These points are created in the $[0, 1]^d$ hypercube and then mapped to the hyperparameter space. Each point is evaluated, then the search space is halved along each dimension and centered at the best point. New points are added to this reduced space, again spread evenly taking the existing points into consideration as well. This is then repeated for a pre-specified number of number of iterations. To make things clearer, consider the following plots which show as example of this in two dimensions.



Now that we have that image of the algorithm in our minds, here is the pseudo code.

Algorithm 5 Sequential Uniform Designs

```
1: procedure SEQUD( $n\_points$ ,  $T_{max}$ ,  $bounds$ ,  $\mathcal{D}_{train}$ )
2:    $r_b \leftarrow 0.5$ 
3:    $U \leftarrow$  empty set
4:   for  $t \in \{1, \dots, T_{max}\}$  do
5:     if  $t = 1$  then
6:        $T \leftarrow create\_UD(n\_points)$ 
7:     else
8:        $r_b \leftarrow \frac{r_b}{2}$ 
9:        $lwr \leftarrow best - r_b$ 
10:       $upr \leftarrow best + r_b$ 
11:       $box\_points \leftarrow \{U : lwr < U_i < upr\}$ 
12:       $T \leftarrow augmentUD(n\_points, box\_points, lwr, upr)$ 
13:    end if
14:    for  $i \in \{1, \dots, n\}$  do
15:       $L_i \leftarrow objective\_function(\mathcal{D}_{train}, T_i)$ 
16:    end for
17:     $best \leftarrow T_{which.min(L)}$ 
18:  end for
19:  return Parameter configuration with the lowest error.
20: end procedure
```

Considerations

Strengths:

- Maximizes exploration of the search space for a given number of configs. Avoids checking the same or very similar configs twice.
- By searching around the previous best it acts similar to Bayes search, making it less likely to miss good configs in spaces between configs like the non-sequential searchers.
- Can still be run in parallel for config evaluations, unlike Bayes.

Weaknesses:

- If the objective function has two ‘peaks’ only one may be explored in the subsequent iteration, which may result in the best model being missed.
- Needs some intuition to use: how to balance iterations vs number of points per iteration.
- Augmenting the UD can take some time, but usually not as much as Bayes.

Extensions:

- Instead of zooming in to just one point we could zoom in on multiple if the second best configuration is not in the new box. We could also use an adaptive size for the new box, removing the ‘bad’ parts of the search space first.
- As with all algorithms we could also experiment with limited resource allocation to make it run faster.
- If augmentation takes longer than function evaluation, then one could not augment, but use the original design shrunk down to the size of the new box. This may result in similar models being checked twice but it will still be faster. Additionally we could also just remove similar configurations if they occur, in which case it will always be faster.
- One could also use the same distributions as in Random Search. The $[0, 1]^d$ hypercube can be converted to the (possibility joint) multivariate distribution, by use of the inverse CDF. This way prior knowledge of the parameters can be used to more effectively search the space.

R code: [Click Me!](#)

Evolution Strategies (ES)

Not tested in this project, however this class of algorithms has been shown to be effective and is worth mentioning. These algorithms are inspired by biological evolution. They follow this general structure:

- 1. Select random starting config.
- 2. Evaluate the selected configs.
- 3. Create new configs by ‘combining’ or ‘mutating’ the best configs.
- 4. Return to step 2 until some stopping criterion are met.

So each ‘generation’ would theoretically be a potential improvement to the previous one. There is a lot of flexibility in how this algorithm is formulated and what the ‘mutation’ and ‘combination’ functions look like. In that way it makes more sense to apply this class of algorithms to the neural architecture search problem. Where it can be difficult to formulate the other algorithms, as we will see they can be rather constrained as the parameters generally have to be in a fixed parameter space, where as in ES algorithms that is not a limitation. This class of algorithms could more than fill another project of this size all on their own, for our purposes we will go no further into ES.

Algorithm Testing

Experiment Setup

In the experiment four different objective functions are tested. When given a hyperparameter configurations they return a corresponding validation error. The different models and corresponding search space dimensions are:

Model	Number of Parameters
Random Forest	2
XGBoost	4
Feed Forward Neural Network	10, 13
Convolutional Neural Network	16

As mentioned in the introduction, this is a very limited experiment due to a lack of the computational resources to test these methods properly (It would takes months to run on my computer). As such we cannot draw any conclusive results from this. However, this is a decent starting place to identify some strengths and weaknesses of each algorithm, and possibly some extensions that may help in a practical application.

The datasets used were taken from the UCI Machine Learning Repository. Each model, except for CNN was tested on the following five datasets:

- Heart Disease [<https://archive.ics.uci.edu/dataset/45/heart+disease>]
- Bank Marketing [<https://archive.ics.uci.edu/dataset/222/bank+marketing>]
- Forest Fire [<https://archive.ics.uci.edu/dataset/162/forest+fires>]
- Bike Sharing [<https://archive.ics.uci.edu/dataset/275/bike+sharing+dataset>]
- Superconductivity [<https://archive.ics.uci.edu/dataset/464/superconductivty+data>]

CNN was tested only on MNIST, again due to computational limitations.

Models are selected by comparing the validation error defined as follows for each model:

- Random Forest: Out-of-Bag (OOB) Error
- XGBoost: 3-fold Cross Validation
- Neural Network: Train / Valid split

Each optimizer will return the validation and test errors of the selected configuration, as well as the runtime. The runtime includes the full run of the algorithm up-to and including the selection of the best configuration, it excludes the calculation of the test error.

The resources for the algorithms were assigned as follows:

Random Search: The same number of configurations as Grid Search.

Bayes Search: Time limit 10% more time than Grid Search for the same model and dataset, some exceptions in the case that Grid Search ran too fast.

Hyperband: Roughly set budget so that it takes a similar time to Grid Search.

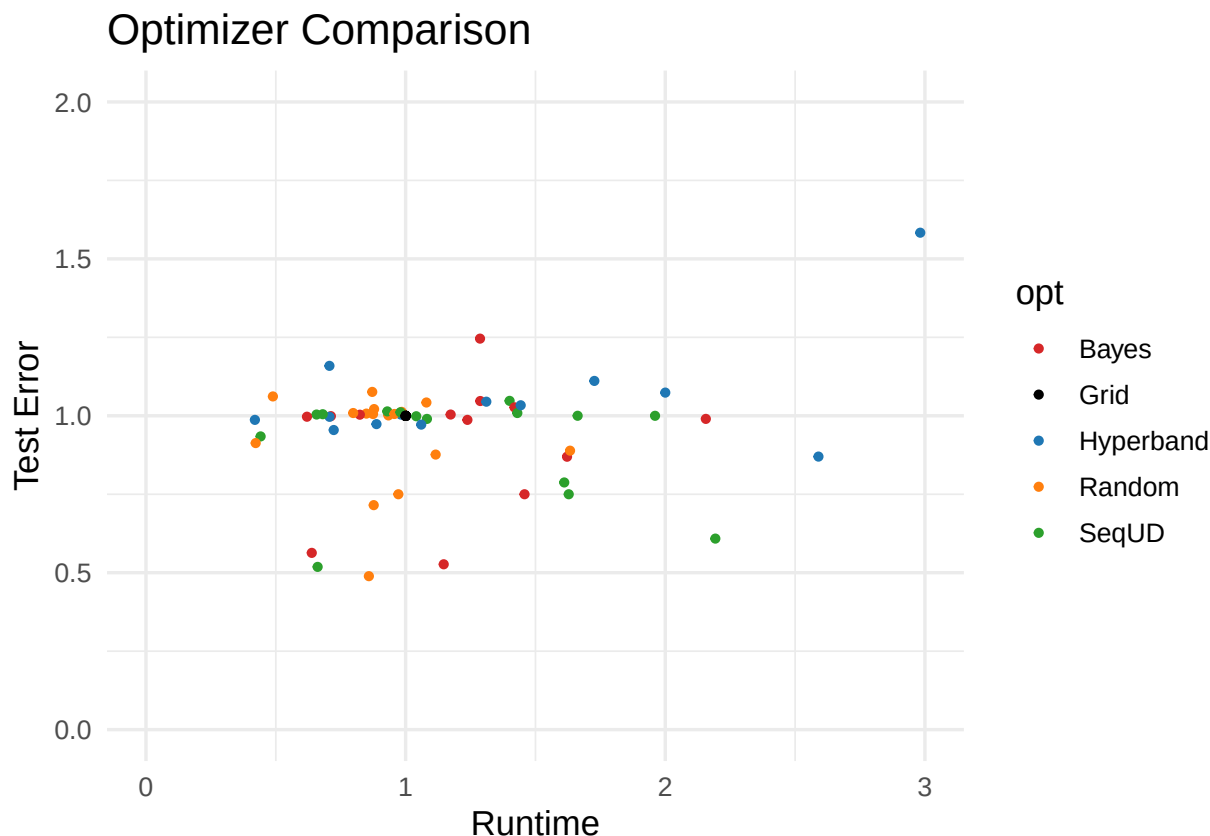
SeqUD: Same as Hyperband.

Preliminary Results

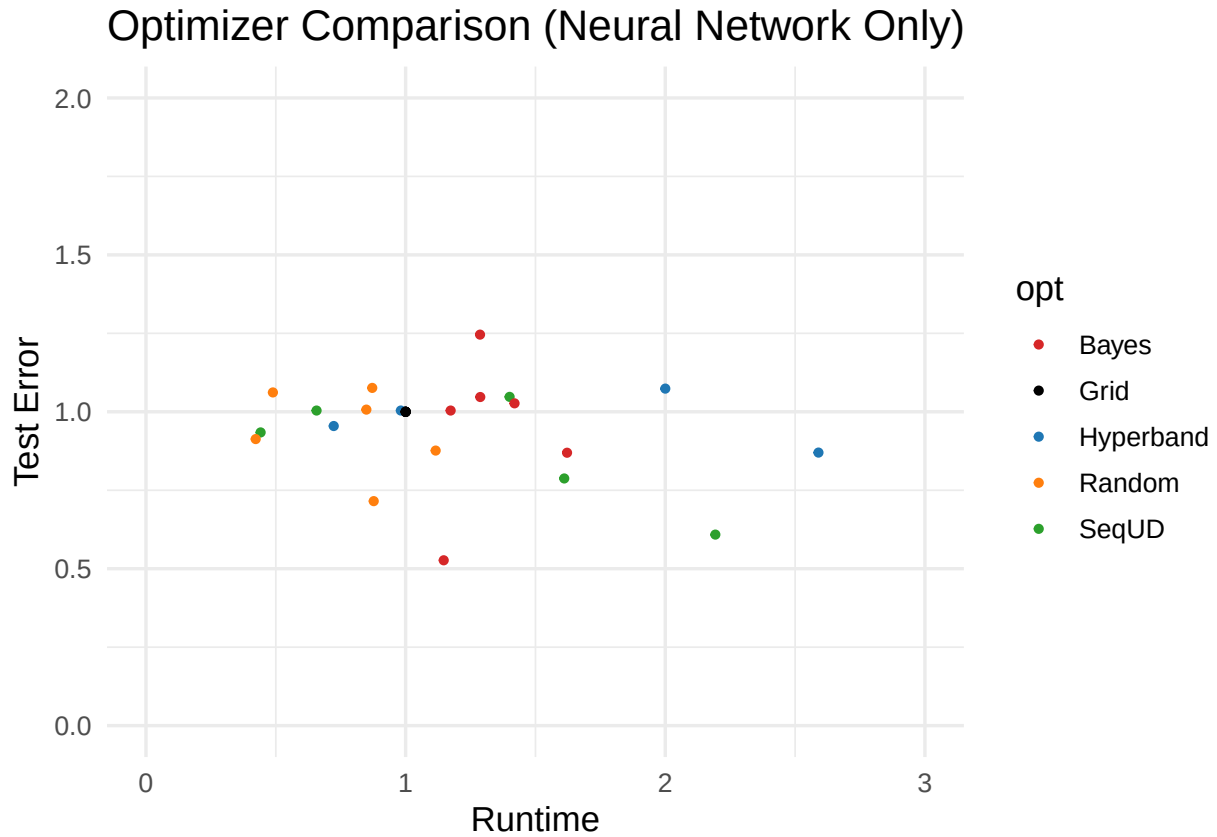
There are 16 runs per optimizer for a total of 80 observations. As stated before there are not enough observations to draw conclusions. Furthermore, the best optimizer changes at different dimensions. For example if the objective function can be evaluated quickly then Bayes and SeqUD are at a disadvantage since they spend time picking the next point.

The runtimes and test errors were normalized by dividing by the runtime and test error of Grid Search for that particular dataset and model.

The relative runtimes and test errors are in the plot below:



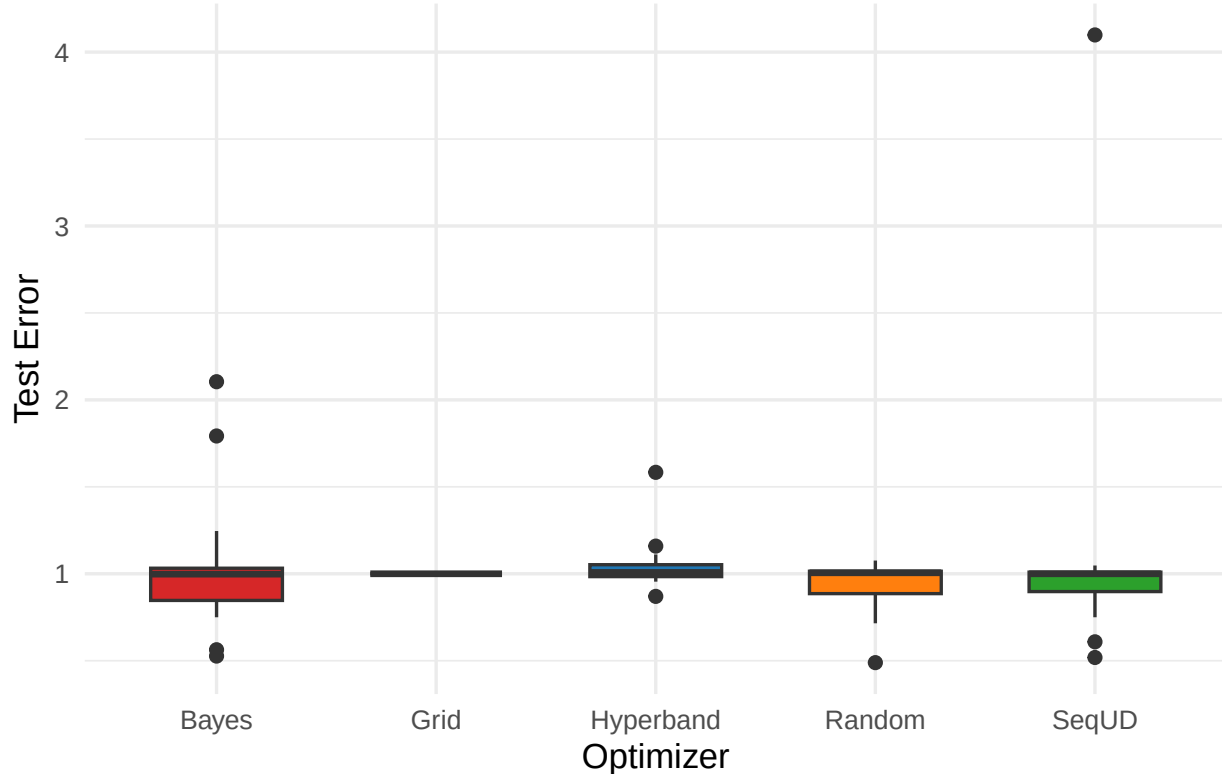
One thing that we can see is that Random Search seems to run faster than Grid Search. This could be because each model has a parameter that controls how long the objective function takes to run, for example tree depth or learning rate. On average, Random Search distributions chose faster configurations than Grid Search. So it is of course dependent on the grid, as we would expect Random Search to have the same runtime. It may be that making the Grid takes longer than randomly drawing, however I do not believe that is a noticeable difference. We do not see any clear patterns with the other optimizers. Let's take a look at just the neural network runs. As generally HPO is used just for neural networks and the patterns may be clearer at higher runtimes and more complex problems.



Bayesian Optimizer takes a very long time to select the next point, for the CNN it took about 5-8 minutes to evaluate one configuration, but Bayes might take 60-120 minutes to pick the next point. So it tests comparatively very few configurations, which is likely why it performs worse than the others here.

The runtimes were intentionally meant to be similar, so let's focus on the distribution test errors.

Test Error Distribution by Optimizer



This plot gives the impression that Hyperband under performs. However, again, we do not have enough data to draw clear results. There are some observations that can be made. The sequential algorithms (Bayes and SeqUD) do better in lower dimensions and when the objective function takes longer to run. The longer the objective function takes to run the smarter we want to be about what configuration to try, but as dimensionality increases these algorithms take significantly longer to pick new points. If we can pick good distributions for Random Search it works very well. Never use Grid Search. Hopefully this was interesting to you! Thank you for reading.

Appendix

R Code

Grid Search Code

[Click here to go back](#)

```
grid_search <- function(X, y, obj_func, param_values) {  
  # Create the grid  
  grid <- expand.grid(param_values, stringsAsFactors = FALSE)  
  
  # Evaluate objective function at each point  
  n <- nrow(grid)  
  errors <- numeric(length = n)  
  for(i in 1:n) {
```

```

    errors[i] <- obj_func(X, y, params=grid[i,])
  }

  # find & return best model (lowest error)
  return(grid[which.min(errors), ])
}

```

Random Search Code

[Click here to go back](#)

```

random_search <- function(X, y, obj_func, param_dists, n) {
  # get n many realizations of the parameters
  grid <- data.frame(row.names = 1:n)
  for(p in names(param_dists)) {
    grid[[p]] <- param_dists[[p]](n)
  }

  # Evaluate objective function at each point
  errors <- numeric(length = n)
  for(i in 1:n) {
    errors[i] <- obj_func(X, y, params=grid[i,])
  }

  # find & return best model (lowest error)
  return(grid[which.min(errors), ])
}

```

Bayesian Optimization Code

[Click here to go back](#)

This one is really simple since the bayesOpt function does all the work for us.

```

bayes_search <- function(X, y, obj_func, param_bounds, n, t_limit=NULL) {
  scoreFunction <- function(...){
    # negative since bayesOpt is a maximizer
    list(Score = -obj_func(X=X, y=y, params=list(...)))
  }
  init_points <- max(6, length(param_bounds)+1)

  optObj <- ParBayesianOptimization::bayesOpt(
    FUN = scoreFunction,
    bounds = param_bounds,
    initPoints = init_points, # randomly eval at some points initially
    iters.n = n,
    iters.k = 1,
    otherHalting = list(timeLimit = t_limit)
  )
  # Return Results
  return(data.frame(getBestPars(optObj)))
}

```

Hyperband Code

[Click here to go back](#)

```
hyperband <- function(X, y, obj_func, param_dists, R, eta = 3) {
  all_results <- list()

  # maximum number of brackets
  s_max <- floor(log(R, base = eta))
  B <- (s_max + 1) * R

  for(s in s_max:0) {
    n <- ceiling((B / R) * eta^s / (s + 1))
    r <- R * eta^(-s)

    # Begin Successive Halving in inner loop
    # get n many realizations of the parameters
    grid <- data.frame(row.names = 1:n)
    for(p in names(param_dists)) {
      grid[[p]] <- param_dists[[p]](n)
    }

    for(i in 0:s) {
      n_i <- nrow(grid)
      r_i <- r * eta^i

      # Get losses
      L <- numeric(length = n_i)
      for(j in 1:n_i) {
        # r_frac is normalized resource allocation
        L[j] <- obj_func(X, y, params = grid[j, ], r_frac = r_i/R)
      }

      grid$error <- L
      all_results[[length(all_results) + 1]] <- grid

      # Update grid to be just the top n_i / eta performers
      k <- max(1, floor(n_i / eta))
      grid <- head(grid[order(grid$error), ], k)
    }
  }
  # Combine everything
  results <- do.call(rbind, all_results)
  # Find & Return best model
  return(results[which.min(results$error), ])
}
```

SeqUD Code

[Click here to go back](#)

```
augmentUD <- function(X0, add, n_candidates = 1000) {
  # X0: existing design (rows = runs, cols = factors), scaled to [0,1]
```

```

# add: number of new points to add
# n_candidates: number of candidate points to try each step
d <- ncol(X0)
new_points <- c()

for (k in 1:add) {
  # generate random candidate points
  C <- matrix(runif(n_candidates * d), ncol = d)
  # score each candidate by discrepancy when added
  scores <- apply(C, 1, function(cand) {
    SFDesign::uniform.crit(rbind(X0, cand))
  })
  # pick best candidate
  best <- C[which.min(scores), , drop = FALSE]
  # update design
  X0 <- rbind(X0, best)
  new_points <- rbind(new_points, best)
}
new_points
}

seqUD <- function(X, y, obj_func, param_bounds, T_max, n_points){
  d <- length(param_bounds)
  # Find range and min of each param
  mins <- sapply(param_bounds, function(x) as.numeric(x[1]))
  lwr <- mins
  upr <- sapply(param_bounds, function(x) as.numeric(x[2]))
  ranges <- sapply(param_bounds, function(x) as.numeric(diff(x)))
  is_int <- sapply(param_bounds, function(x) is.integer(x))

  from_01_to_ps <- function(U) {
    out <- matrix(NA, nrow = nrow(U), ncol = d)
    for (j in 1:d) {
      vals <- mins[j] + U[, j] * ranges[j]
      if (is_int[j]) {
        vals <- round(vals)
      }
      out[, j] <- vals
    }
    colnames(out) <- names(param_bounds)
    as.data.frame(out)
  }

  U01 <- c()
  grid <- c()
  box_radius <- 0.5

  for(t in 1:T_max) {
    # Generate points
    if(length(U01) == 0){ # Make fresh design
      new_points <- uniformLHD(n = n_points, p = d)$design
      # map to parameter space
      ps_points <- from_01_to_ps(new_points)
    }
  }
}

```

```

} else { # Augment existing
  # update boundaries
  box_radius <- box_radius * 0.5
  lwr <- best - box_radius
  upr <- best + box_radius

  # Check if the box is outside [0,1]^d
  shift_lwr <- pmax(0 - lwr, 0) # how far below 0
  shift_upr <- pmin(1 - upr, 0) # how far above 1 (negative)

  # total shift = whichever adjustment is needed
  shift <- shift_lwr + shift_upr

  # apply shift to corners
  lwr <- lwr + shift
  upr <- upr + shift

  # Find which points are in the new bounds
  in_the_box <- apply(U01, MARGIN=1, FUN = function(x) {2 * d == sum(c(x > lwr, x < upr ) )})
  box_points <- U01[in_the_box, , drop=FALSE]

  # Augment the sub design
  n_e <- n_points - nrow(box_points)
  new_points <- augmentUD(sweep(box_points, 2, lwr, "-") / (box_radius * 2), add = n_e, n_candidates)

  # Shift the design back into place
  new_points <- (new_points * (box_radius * 2))
  new_points <- sweep(new_points, 2, lwr, "+")

  # Generate points in the parameter space
  ps_points <- from_01_to_ps(new_points)
}

# Evaluate each point
n <- nrow(ps_points)
errors <- numeric(length = n)

for(i in 1:n) {
  errors[i] <- obj_func(X, y, params=ps_points[i, ])
}

# Save the errors
ps_points$error <- errors
# Append new points onto old points
grid <- rbind(grid, ps_points)
U01 <- rbind(U01, new_points)
# Find the best point
best_idx <- which.min(grid$error)
best <- U01[best_idx, ]
}

# Return results
return(grid[best_idx, ])
}

```


References

- Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13(10), 281-305. Retrieved from <http://jmlr.org/papers/v13/bergstra12a.html>
- Li, L., Jamieson, K., DeSalvo, G., Rostamizadeh, A., & Talwalkar, A. (2018). Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185), 1-52. Retrieved from <http://jmlr.org/papers/v18/16-558/16-558.pdf>
- Yang, Z., & Zhang, A. (2021). Hyperparameter optimization via sequential uniform designs. *Journal of Machine Learning Research*, 22, 1-47. Retrieved from <http://jmlr.org/papers/v22/20-058.html>
- Bischl, B., Binder, M., Lang, M., Pielok, T., Richter, J., Coors, S., Thomas, J., Ullmann, T., Becker, M., Boulesteix, A.-L., Deng, D., & Lindauer, M. (2023). Hyperparameter optimization: Foundations, algorithms, best practices, and open challenges. *WIREs Data Mining and Knowledge Discovery*, 13(2), e1484.
- Garnett, R. (2023). *Bayesian optimization*. Cambridge University Press. ISBN 978-1-108-42578-0.